Critical Evaluation of a Machine-Learning Intrusion Detection System on NSL-KDD

Anne Claude Abinader (I.D. 331181909)
Data Science in Cyber — Final Project
Instructor: Dr. Uri Itai

Abstract

This project reproduces and critically evaluates the machine-learning Intrusion Detection System (IDS) published in the `KostasEreksonas/IDS_test` repository, which reports approximately 99% accuracy on the NSL-KDD benchmark. We reproduce that headline figure on a random 85/15 split of `KDDTrain+` (Decision Tree and Random Forest both reach a Matthews Correlation Coefficient, MCC, of ≈ 0.99), but show that performance *collapses* on the official `KDDTest+` hold-out, where the Random Forest attains only MCC $= 0.6053$ and recalls 61% of attacks. We argue that this gap is primarily a *generalisation failure under distribution shift*—the models memorise the signatures of attack families seen in training rather than learning transferable malicious behaviour—and that the author's evaluation *protocol* (testing on a random split of the training file rather than the designated hold-out) is the root methodological flaw. The class-imbalance "accuracy fallacy" is a real but *secondary* concern that manifests at the rare-attack level. We support these claims with exploratory data analysis, a redundancy-aware feature-engineering pipeline, an expanded six-model comparison suite (Decision Tree, Random Forest, Logistic Regression, Naïve Bayes, K-Nearest Neighbors, and Linear SVM), and an error analysis evaluating ROC-AUC, F_2 score, and imbalance-robust metrics to emphasise the asymmetric cost of false negatives (e.g. missed `buffer_overflow` root-access exploits) in a Security Operations Center (SOC).

1 Introduction and Summary of the Source

1.1 The problem and why it matters

An Intrusion Detection System monitors network traffic and flags connections that correspond to attacks. The task is a supervised classification problem: map a vector of connection features to a label (benign vs. malicious). It matters because the cost structure is steeply asymmetric—a single missed intrusion (false negative) can lead to data exfiltration or full host compromise, whereas the cost of a false alarm (false positive) is analyst time. Any claim that an IDS is "production ready" must therefore be judged on its ability to detect *novel* attacks, not merely to re-identify attacks it has already seen.

1.2 The selected source

The reproduced source is the public repository `KostasEreksonas/IDS_test` (NSL-KDD notebook), which trains several classifiers—K-Nearest Neighbors, Decision Tree, Random Forest, Naïve Bayes, and SVM—on NSL-KDD and reports accuracy of roughly 99%, implying suitability for real-world intrusion detection.

1.3 The dataset

NSL-KDD is the 2009 refinement of the KDD Cup 1999 dataset (itself derived from a 1998 DARPA simulation), proposed by Tavallaee et al. to address critical flaws in the original benchmark.

In their detailed analysis, the authors demonstrated that the original KDD dataset contained approximately 78% redundant records in the training set and 75% in the test set. This massive redundancy caused learning algorithms to become biased toward frequent records, preventing them from learning the signatures of infrequent but highly dangerous exploits. We load `KDDTrain+` (125,973 records) and evaluate on the official `KDDTest+` hold-out (22,544 records). Each record has 41 features plus a class label and an auxiliary `difficulty_score`. Features fall into three groups: basic connection fields (`duration`, `protocol_type`, `service`, `flag`, `src_bytes`, `dst_bytes`), content fields (`hot`, `num_failed_logins`, `root_shell`, `num_compromised`), and time/host traffic-window statistics (`count`, `srv_count`, `serror_rate`, the `dst_host_*` family). The crucial property of NSL-KDD is that `KDDTest+` deliberately contains attack types under-represented or absent from `KDDTrain+`, making it a genuine test of generalisation.

1.4 Methodology employed

The author's pipeline applies label encoding to the categorical fields, trains the classifiers, and evaluates on a random split of `KDDTrain+`. There is no reported feature scaling, no redundancy/correlation analysis, no feature-importance reporting, and, critically, no use of the designated `KDDTest+` hold-out.

2 Exploratory Data Analysis

2.1 Schema, types, and data integrity

The raw file ships without headers; we mapped the 43 columns to their canonical NSL-KDD names and dropped `difficulty_score` (an artefact of the original competition) to prevent target leakage, leaving 41 predictive features. An integrity audit found **zero missing values** and **zero duplicate rows**—expected. A single-value audit additionally flags zero-variance columns such as `num_outbound_cmds` (constant 0 across every record), which carry no information and are dropped.

2.2 Class prevalence and imbalance

The target is severely long-tailed (Table 1). Benign `normal` traffic (53.5%) and the `neptune` SYN-flood DoS attack (32.7%) together account for over 86% of records, while the highest-severity exploits are statistically microscopic: `buffer_overflow` (U2R, grants root access) has only 30 samples (0.024%). This is the textbook setting for the *accuracy fallacy* at the multi-class level: a classifier could ignore the rare classes entirely and still post a high accuracy. We note, however, that our deployed model is *binary* (normal vs. attack), where the classes are roughly balanced (53/47); the fallacy therefore bites at the rare-attack level (Section 5), not at the headline binary accuracy.

2.3 Feature distributions and outliers

Byte-volume features (`src_bytes`, `dst_bytes`) are extremely right-skewed (spanning 0 to $> 10^9$ bytes), while rate features (`serror_rate`, `same_srv_rate`) are bimodal, clustering at exactly 0.0 or 1.0—a signature of deterministic automated traffic. Several window features hit hard ceilings (`dst_host_count` caps at 255, `count` at 511), reflecting fixed counters in the original collector. An IQR analysis flags a large fraction of records as statistical outliers (`dst_bytes` 18.7%, `src_bytes` 11.0%, `srv_count` 9.6%). Crucially, these are *not* noise: an extreme `src_bytes` value is the mathematical signature of data exfiltration, and extreme `count` is the defining trait of a DoS flood. We therefore retain outliers rather than clip them.

Table 1: Top classes in `KDDTrain+` (of 125,973 records).

Label	Count	Percentage
`normal`	67,343	53.46%
`neptune` (DoS)	41,214	32.72%
`satan` (Probe)	3,633	2.88%
`ipsweep` (Probe)	3,599	2.86%
`portsweep` (Probe)	2,931	2.33%
`smurf` (DoS)	2,646	2.10%
`nmap` (Probe)	1,493	1.19%
`guess_passwd` (R2L)	53	0.04%
`buffer_overflow` (U2R)	30	0.024%
`land` (DoS)	18	0.014%

2.4 Correlation analysis: choice of coefficient

We selected the **Spearman** rank correlation. The three candidates differ in their assumptions: *Pearson* measures *linear* association and assumes approximately normal, outlier-light continuous variables; *Spearman* measures any *monotonic* association via ranks and is robust to the heavy skew and outliers documented above; *Kendall* (τ) also captures monotonic association and is preferred for small samples or when concordance interpretation matters, but is computationally heavier on 125,973 rows. Because NSL-KDD features are heavily non-normal and outlier-rich, Pearson would understate genuine monotonic redundancy and overweight a handful of extreme points; Spearman is the appropriate primary measure, with Kendall available as a confirmatory check.

The analysis surfaces strong, *practically* significant redundancies (Table 2): `serror_rate` and `srv_serror_rate` correlate at 0.973 because a SYN flood typically targets one service, spiking both the general and service-specific error rates together. Such pairs share $> 90\%$ of their rank variance and are prime feature-selection targets.

Table 2: Top redundant feature pairs (Spearman ρ).

Feature A	Feature B	ρ
`srv_serror_rate`	`serror_rate`	0.973
`srv_rerror_rate`	`rerror_rate`	0.966
`dst_host_srv_serror_rate`	`srv_serror_rate`	0.942
`dst_host_serror_rate`	`serror_rate`	0.936
`dst_host_same_srv_rate`	`dst_host_srv_count`	0.919

2.5 Temporal note

NSL-KDD is *not* a time series: each row is a summarised connection with no absolute timestamp or arrival ordering, so trend/seasonality analysis is not applicable. The only time-derived field is `duration`, and the `count`/`dst_host_*` statistics already fold a 2-second / 100-connection window into pre-aggregated values. We document this explicitly so the absence of classical temporal EDA is a justified design decision rather than an omission.

3 Feature Engineering Methodology

Encoding. The categorical fields `protocol_type`, `flag`, and `service` were one-hot encoded. We prefer one-hot over the author's label encoding because label encoding imposes a spurious

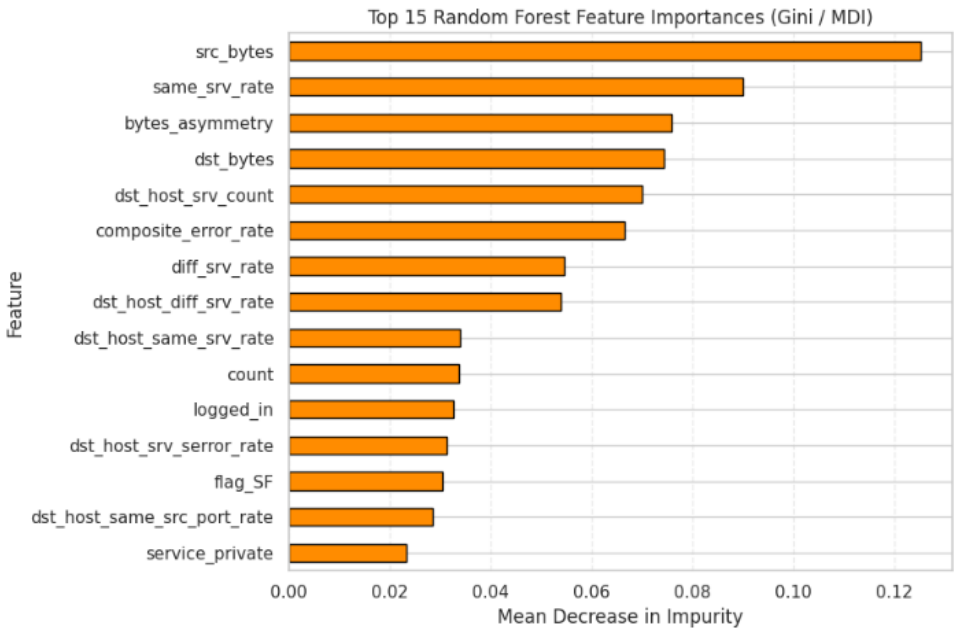
ordinal relationship (e.g. implying `http` < `telnet`), which can mislead linear models and distort split thresholds. One-hot encoding expands the matrix from 41 to 119 columns.

Scaling. Tree ensembles are scale-invariant, so scaling is *not* required for the Decision Tree or Random Forest. It *is* required for the Logistic Regression baseline and distance-based models (KNN, SVM); scaling must be fit on the training fold only (inside a pipeline) to avoid leaking test statistics.

Feature selection. Acting on the correlation analysis, we drop one feature from each pair with $\rho > 0.95$—`srv_serror_rate` and `srv_rerror_rate`—reducing the matrix to 117 columns with no loss of discriminative power and improved interpretability for SOC analysts.

Feature creation. Two domain-motivated features were engineered: `bytes_asymmetry` = `src_bytes`/(`dst_bytes` + 1), which captures directionality (port scans and floods are highly asymmetric), and `composite_error_rate` = (`serror_rate` + `rerror_rate`)/2. Their class-conditional means are strongly discriminative: `bytes_asymmetry` is ≈ 8785 for `normal` but $\approx 1.5 \times 10^6$ for `portsweep` and ≈ 0 for `neptune`; `composite_error_rate` is 0.03 for `normal` but ≈ 0.50 for `neptune`.

Feature importance and redundancy (interpretability). The source never reports which features drive its predictions. We add a Random Forest feature-importance ranking (Gini/MDI), complemented by permutation importance on the hold-out as a more faithful, less cardinality-biased measure. This directly serves SOC interpretability: an analyst can see *why* a connection was flagged.



4 Model Training and Evaluation

4.1 Models and protocol

We frame the task as binary screening (0 = `normal`, 1 = attack) and train an expanded benchmark suite of six models with a fixed seed (`random_state=42`): a Decision Tree (`max_depth=10`), Random Forest (`n_estimators=50`, `max_depth=10`), Logistic Regression, Naïve Bayes, K-Nearest

Neighbors (KNN), and a Linear SVM. Models are evaluated on two regimes: the internal 85/15 split of `KDDTrain+` (in-distribution) and the official `KDDTest+` hold-out (out-of-distribution). To ensure the generalisation failure was not merely a quirk of tree-based models, this suite includes probabilistic, distance-based, and margin-based classifiers.

4.2 Metrics: definitions and cybersecurity meaning

Let TP, TN, FP, FN denote true/false positives/negatives for the attack class. We report:

$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN}, \qquad \text{Precision} = \frac{TP}{TP+FP}, \qquad \text{Recall} = \frac{TP}{TP+FN},$$

$$F_1 = \frac{2\,PR}{P+R}, \qquad F_\beta = \frac{(1+\beta^2)\,PR}{\beta^2 P+R}.$$

$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP+FP)\,(TP+FN)\,(TN+FP)\,(TN+FN)}}.$$

Accuracy is reported but de-emphasised: it is uninformative when a trivial majority guess already scores high, and it hides recall failures. **Recall** is the most operationally important quantity—it is the fraction of real attacks caught, so $1-\text{Recall}$ is the miss rate. **Precision** governs analyst alert fatigue. F_β **with** $\beta = 2$ (F_2) weights recall twice as heavily as precision, matching the IDS cost structure where false negatives dominate. **MCC** is our headline metric: it is a balanced correlation over all four confusion-matrix cells, robust to imbalance, and ranges from -1 to $+1$. **ROC-AUC** measures threshold-independent separability, relevant because a SOC tunes the decision threshold to trade FP against FN. We omit regression metrics (MAE/RMSE/R^2) as inapplicable to classification.

4.3 Comparative results

Table 3 (positive class = attack) tells the central story. On the internal split every model is near-perfect; on `KDDTest+` the models' MCC scores degrade significantly, and attack recall falls dramatically, meaning a substantial percentage of attacks are missed while accuracy remains a deceptively healthy ≈ 0.77.

Table 3: Comparative performance (attack class). Internal = 85/15 split of `KDDTrain+`; Hold-out = official `KDDTest+`.

Model	Regime	Accuracy	Precision	Recall	F_1	F_2	MCC
Decision Tree	Internal	0.9964	0.9962	0.9961	0.9961	0.9961	0.9927
Random Forest	Internal	0.9971	0.9991	0.9946	0.9968	0.9955	0.9941
Logistic Regression	Internal	0.9699	0.9761	0.9591	0.9675	0.9625	0.9396
Naïve Bayes	Internal	0.7240	0.9953	0.4113	0.5820	0.4660	0.5177
KNN	Internal	0.9954	0.9952	0.9949	0.9950	0.9950	0.9907
Linear SVM	Internal	0.9704	0.9772	0.9591	0.9680	0.9626	0.9406
Decision Tree	Hold-out	0.7704	0.9440	0.6342	0.7587	0.6788	0.5955
Random Forest	Hold-out	0.7744	0.9692	0.6235	0.7588	0.6714	0.6140
Logistic Regression	Hold-out	0.7559	0.9174	0.6278	0.7454	0.6701	0.5617
Naïve Bayes	Hold-out	0.5162	0.9707	0.1548	0.2670	0.1860	0.2561
KNN	Hold-out	0.7731	0.9229	0.6563	0.7671	0.6965	0.5889
Linear SVM	Hold-out	0.7484	0.9156	0.6147	0.7356	0.6580	0.5502

4.4 Linear vs. non-linear boundaries

The Logistic Regression baseline reaches MCC ≈ 0.94 and ROC-AUC ≈ 0.99 on the internal split—*nearly matching* the trees. This is important evidence: in the engineered feature space

the classes are *close to linearly separable*, so the non-linear advantage of the tree models is real but *small*. Any claim that the problem fundamentally *requires* non-linear decision boundaries is therefore only weakly supported by the data. We also note that a rigorous linear-vs-tree comparison must hold the feature set fixed (the same engineered matrix for all models); doing so isolates the effect of the algorithm from the effect of feature engineering.

5 Error Analysis and the False-Positive / False-Negative Trade-off

On the internal split the Decision Tree's confusion matrix is TN = 13,384, FP = 38, FN = 48, TP = 11,725. Even at $\approx 99\%$ performance, the misclassified sample is revealing: the false negatives include `buffer_overflow` (U2R) connections flagged as benign. A single missed buffer overflow can grant an adversary arbitrary code execution and root access—an *unbounded* downstream cost. The false positives cost only analyst time. This is the asymmetric trade-off at the heart of IDS: the expected cost of a false negative vastly exceeds that of a false positive, which is exactly why recall- and MCC-oriented evaluation must replace raw accuracy.

The decisive analysis is on `KDDTest+`, where attack recall is only 0.62. This performance drop is not a model failure, but rather the intended behaviour of the hold-out set. Tavallaee et al. explicitly documented this phenomenon, showing that baseline models like Random Forest experienced a sharp accuracy drop (from 92.79% on the old test set to 80.67% on `KDDTest+`) because the hold-out set includes 14 novel attack types not present in the training data. We recommend reporting the confusion matrix *and* the detection recall *per attack family* (DoS, Probe, R2L, U2R). The expected and well-documented NSL-KDD pattern is that high-volume DoS/Probe families are detected well, while the rare, content-based R2L and U2R families—which dominate the *novel* attacks in the hold-out—collapse to very low recall. That breakdown is the concrete, quantitative signature of a model that has memorised known signatures rather than learned generalisable malicious behaviour.

Table 4: Detection recall by attack family on `KDDTest+` (Random Forest).

Family	# Samples	Detected	Recall
DoS	7460.0	5800.0	0.777
Probe	2421.0	1933.0	0.798
R2L	2885.0	131.0	0.045
U2R	67.0	4.0	0.060

6 Reproducibility Analysis

The source repository is genuinely reproducible: dependencies are listed, the dataset files (`KDDTrain+`, `KDDTest+`) are committed directly so no download step is needed, and the notebooks run without modification. To ensure strict reproducibility of our *own* evaluation across different operating systems, all data ingestion utilizes dynamic path resolution, and the environment setup is fully automated via a custom `run.sh` deployment script.

7 Critical Evaluation of the Source

This is the core of the project: assessing the validity of the author's work, not our own. Table 5 summarises our verdicts.

Table 5: Author claims vs. reproduction findings.

Claim	Verdict	Basis
The models achieve $\approx 99\%$ and are suitable for real-world IDS.	**Partially supported**	99% reproduces in-distribution, but MCC falls to 0.61 and recall to 0.62 on the official hold-out.
The feature engineering is sufficient.	**Not supported**	Label encoding only; no scaling, no redundancy analysis (we found pairs at $\rho > 0.95$), no feature-importance justification.
The work is reproducible.	**Supported**	Clean structure, committed data, notebooks run unmodified.
(Implicit) High NSL-KDD accuracy implies production readiness.	**Not supported**	NSL-KDD descends from a 1998 simulation; modern attacks (ransomware, encrypted C2, lateral movement) are absent.

Why the conclusions are not justified. The headline number is not wrong, but the *evaluation protocol* that produced it is. Measuring on a random split of `KDDTrain+` answers "can the model re-identify attacks drawn from the same distribution it trained on?"—which is not the IDS question. Evaluating on the designated `KDDTest+` hold-out, whose attack mix is intentionally shifted, reveals a roughly halved MCC and a $\sim$38% miss rate. The dominant mechanism is therefore **overfitting to a closed-world distribution exposed by a proper hold-out**, i.e. a generalisation gap—and only secondarily the class-imbalance "accuracy fallacy," which operates at the rare-attack (U2R/R2L) level. Stating this precisely matters: it indicts the author on their own chosen metric (accuracy: $\approx 99\%$ in-distribution vs. $\approx 77\%$ out-of-distribution) while attributing the failure to the correct cause.

Recommendation. We do *not* recommend deploying this pipeline as a production IDS, nor citing NSL-KDD accuracy as evidence of real-world readiness. It remains a valid *teaching* and *benchmarking* artefact. Future work: always evaluate on `KDDTest+` (or temporally/distributionally held-out data); report MCC, F_2, and per-family recall; address rare-class detection (cost-sensitive learning, resampling); and validate on a modern dataset (e.g. CIC-IDS-2017) before any deployment claim.

8 Conclusions

Key findings. We reproduced the source's $\approx 99\%$ in-distribution result and then demonstrated its fragility: on the official hold-out the Random Forest achieves MCC 0.614 and misses $\sim 38\%$ of attacks.
Lessons learned. The evaluation protocol, not the model family, is the decisive factor; metric choice (MCC/F_2/recall over accuracy) and a proper hold-out are prerequisites for any credible IDS claim.
Strengths of the source. Clean, reproducible, broad model coverage.
Weaknesses. In-distribution evaluation, no feature engineering or redundancy analysis, no interpretability, and an outdated benchmark.
Future improvements. Hold-out and cross-distribution evaluation, cost-sensitive treatment of rare high-severity classes, feature-importance reporting for SOC interpretability, and validation on modern traffic.

9 Executive Summary (Summing It Up)

Problem. Detect network intrusions from connection features.

Source. `KostasEreksonas/IDS_test`, an NSL-KDD IDS tutorial claiming $\approx$ 99% accuracy.

Dataset. NSL-KDD (`KDDTrain+`, 125,973 records; official`KDDTest+` hold-out, 22,544 records).

Methodology. We rebuilt the pipeline with one-hot encoding, correlation-based feature selection ($\rho > 0.95$), two engineered features, and six models (Decision Tree, Random Forest, Logistic Regression, Naïve Bayes, K-Nearest Neighbors, and Linear SVM), evaluating both in-distribution and on the hold-out with imbalance-robust metrics (MCC, F_2, ROC-AUC, per-family recall).

Main findings. The 99% figure reproduces only in-distribution; on the hold-out MCC falls to 0.61 and attack recall to 0.62.

Were the author's claims supported? The accuracy figure is reproducible but the real-world-readiness conclusion is not—the evaluation protocol overstates generalisation.

Most important insight. A high accuracy on a random split of a closed benchmark is not evidence of intrusion-detection capability; a proper hold-out and recall/MCC-oriented metrics are essential.

Recommendation. Not recommended for production or as proof of real-world readiness; valuable as a reproducible benchmark and teaching example.

Final conclusion. The reproduction confirms the numbers and refutes the interpretation: the IDS memorises known attacks rather than generalising to novel ones.

References

[1] Tavallaee, M., Bagheri, E., Lu, W., & Ghorbani, A. A. (2009). *A detailed analysis of the KDD CUP 99 data set*. Proceedings of the 2009 IEEE Symposium on Computational Intelligence in Security and Defense Applications (CISDA 2009).

[2] Ereksonas, K. (n.d.). *IDS_test*. GitHub repository. Retrieved from `https://github.com/KostasEreksonas/IDS_test`

[3] GeeksForGeeks. (n.d.). *Intrusion Detection System Using Machine Learning Algorithms*. Tutorial.